

Understanding the Wazuh Indexer

Architecture, Core Features, and Integration
within the Wazuh Platform

A study report on the central data store of the Wazuh SIEM platform

Network Security & Monitoring — George Leonard Batca

September 16, 2026

Written as part of the open-source NSM tools assessment.

Contents

1	Introduction to the Wazuh Indexer	2
2	What the Wazuh Indexer Actually Does	2
2.1	Data Indexing and Storage	2
2.2	Sharding and Replication	3
2.3	Near Real-Time Search	3
2.4	Index Lifecycle Management	3
2.5	Security and Access Control	4
2.6	Purpose-Built Plugins	4
3	Internal Architecture	4
3.1	Cluster Topology and Data Flow	4
4	Limitations	5
5	Deployment Scenarios	5
5.1	AWS Deployment	5
5.2	On-Premises Deployment	6
6	Integration with the Broader Platform	6
7	Conclusion	7

1. Introduction to the Wazuh Indexer

The Wazuh Indexer is one of the three central components of the Wazuh open-source security platform, alongside the Wazuh Server and the Wazuh Dashboard. It is a highly scalable, full-text search and analytics engine built on top of OpenSearch — itself a community-driven fork of Elasticsearch 7.10.2, created after Elastic changed its licensing model.

The Indexer serves as the *memory* of the platform. The Wazuh Server analyzes and correlates security events; the Indexer stores all of that processed data and makes it searchable. Every alert, vulnerability detection event, file integrity change, and inventory record ends up indexed here. The Wazuh Dashboard then queries the Indexer to present that data visually.

The Wazuh Indexer runs on Linux (64-bit Intel, AMD, or ARM) and can be deployed as a single-node instance for development or as a multi-node cluster for production workloads requiring high availability.

2. What the Wazuh Indexer Actually Does

2.1. Data Indexing and Storage

The core function is straightforward: the Indexer takes security data analyzed by the Wazuh Server and stores it in a structured, searchable format. All data is stored as JSON documents, each associating field names with values (strings, numbers, booleans, dates, arrays, geolocations, nested objects). Related documents are grouped into *indices* — purpose-specific collections. The main index types include:

- **wazuh-alerts-*** — Security alerts from the Server’s rule engine (signature matches, anomaly detections, policy violations).
- **wazuh-states-vulnerabilities-*** — Vulnerabilities detected by cross-referencing installed packages against CVE databases.
- **wazuh-states-inventory-packages-*** — Software package inventory from monitored endpoints.
- **wazuh-states-inventory-hardware-*** — Hardware inventory (CPU, memory) from endpoints.
- **wazuh-states-inventory-ports-*** — Open network ports and associated services on endpoints.
- **wazuh-states-inventory-system-*** — System-level details (OS, hostname, architecture) per endpoint.

Key insight

The wildcard (*) in index names reflects time-based suffixes. For example, alert indices are created as `wazuh-alerts-4.x-2025.03.17` for each day. This pattern enables efficient lifecycle policies and time-range queries.

2.2. Sharding and Replication

Documents within an index are distributed across containers called *shards*, which are spread across cluster nodes. This design provides:

- **Redundancy:** Replica shards ensure data survives node failures. In a three-node cluster with one replica, losing a node still leaves a complete copy of every index.
- **Horizontal scalability:** Adding nodes increases parallel query capacity and storage. Shards are automatically redistributed.
- **Automatic rebalancing:** The cluster migrates shards to keep disk usage balanced across nodes.

As a rule of thumb, the number of primary shards should roughly match the number of nodes in the cluster, though this is a sizing guideline rather than a hard requirement. The default installation creates each index with three primary shards and no replicas. Replica count can be adjusted dynamically, but primary shard count cannot be changed without a full re-index — making initial configuration important.

2.3. Near Real-Time Search

The latency between a document being indexed and becoming searchable is typically around one second. For security analysts investigating active incidents, this means data is almost immediately queryable — there is no batch processing delay.

2.4. Index Lifecycle Management

Without management, indices grow until they fill the disk. The Wazuh Indexer uses Index State Management (ISM) policies to automate lifecycle operations:

- **Retention policies:** Automatically delete indices older than a configured period (e.g., 90 days).
- **Rollover policies:** Create new indices when the current one reaches a size or age threshold.
- **Hot-warm architecture:** Recent data lives on fast “hot” nodes; after a configurable period, it migrates to cheaper, slower “warm” nodes.

In any production deployment, ISM policies are essential. Without them, the Indexer will eventually exhaust disk space.

2.5. Security and Access Control

The Security plugin provides role-based access control (RBAC) with predefined Wazuh-specific roles, TLS encryption on both the REST API (port 9200) and the inter-node transport layer (port 9300–9400), and authentication via internal user databases, LDAP, Active Directory, or JWT. All inter-component communication is encrypted by default using certificates generated with Wazuh's `wazuh-certs-tool.sh`.

2.6. Purpose-Built Plugins

The Wazuh Indexer extends OpenSearch with four plugins tailored for security monitoring:

1. **Setup Plugin** — Initializes index templates, ISM policies, data streams, and the Wazuh Common Schema on cluster startup, ensuring correct structure before any data arrives.
2. **Content Manager Plugin** — Synchronizes detection content (rules, decoders, Indicators of Compromise) from the Wazuh Cyber Threat Intelligence (CTI) API, keeping detection capabilities current without manual updates.
3. **Security Plugin** — Provides RBAC, authentication, and TLS encryption.
4. **Reporting Plugin** — Enables on-demand or scheduled generation of PDF/CSV reports from Dashboard visualizations.

3. Internal Architecture

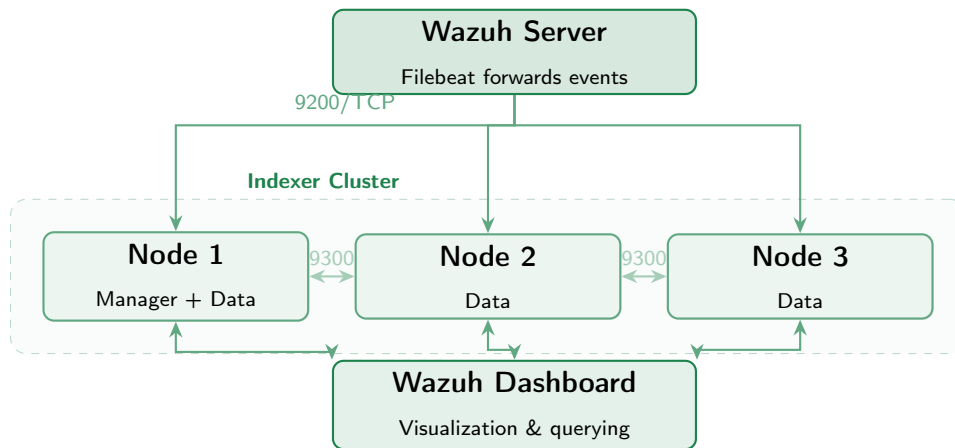
3.1. Cluster Topology and Data Flow

A Wazuh Indexer cluster consists of nodes with distinct roles:

- **Cluster Manager Node** — Manages cluster-wide settings, index metadata, and shard allocation. Can be co-located with a data node in smaller deployments.
- **Data Nodes** — Store data and perform indexing, searching, and aggregation. These need the most disk space and benefit from SSD storage.
- **Ingest Nodes** — Pre-process data through ingest pipelines before storage. Less common in Wazuh deployments since the Server handles most pre-processing.

The data flow follows four stages: (1) the Wazuh Server forwards analyzed alerts and events to the Indexer via Filebeat over port 9200/TCP; (2) the Indexer matches incoming

JSON documents against index templates and writes them to the appropriate shard; (3) primary shards replicate to replica shards on other nodes; (4) the Dashboard or REST clients query the Indexer, with queries distributed across relevant shards.



4. Limitations

The Wazuh Indexer has clear architectural boundaries. It does not analyze raw security data — that is the Server’s job. It does not generate alerts; it only stores them. It does not provide a user interface; the Dashboard is separate. And without ISM policies, indices grow unbounded until disk space runs out. These are not weaknesses but design decisions — the Indexer does one thing well: store and search structured security data at scale.

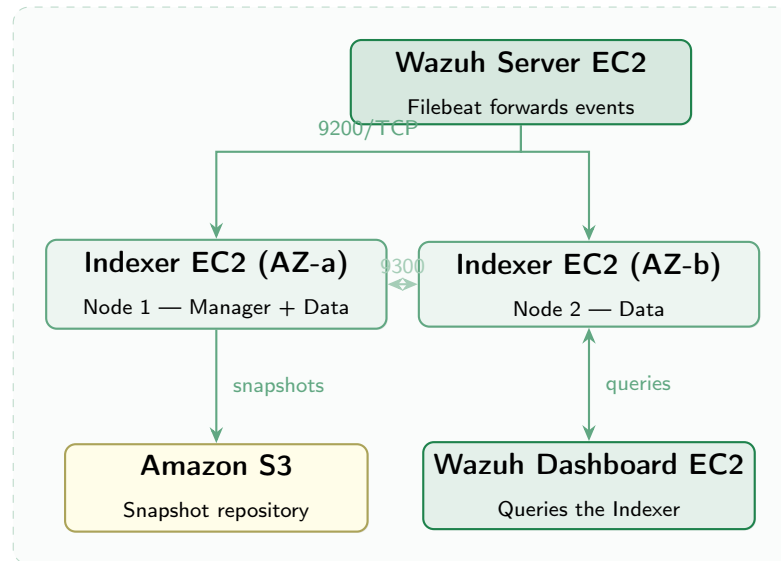
5. Deployment Scenarios

The Indexer can be deployed in cloud or on-premises environments. The core pattern is the same in both: the Wazuh Server forwards events via Filebeat to the Indexer cluster, which stores and indexes them for the Dashboard to query.

5.1. AWS Deployment

In AWS, the Indexer runs on EC2 instances with EBS volumes. A multi-node cluster spread across Availability Zones provides high availability. Snapshot repositories can use S3 for backups and disaster recovery.

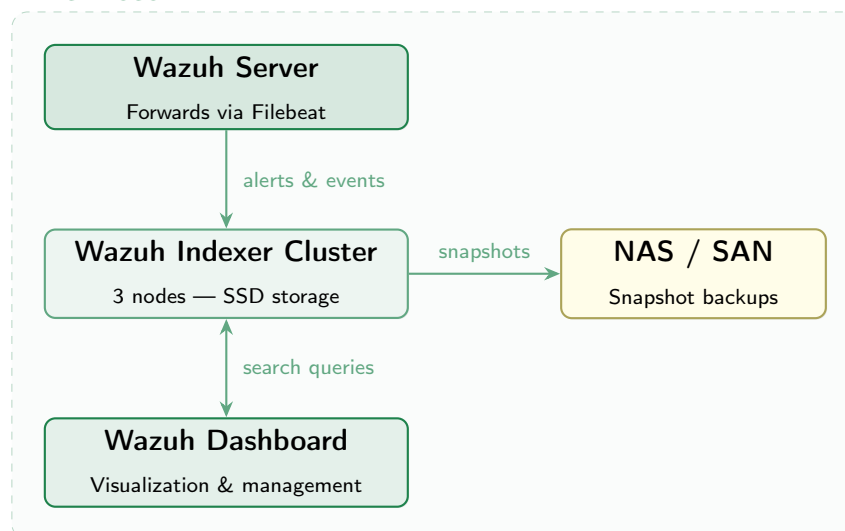
AWS VPC



5.2. On-Premises Deployment

On-premises, the Indexer runs on dedicated physical or virtual servers with local SSD storage. The same multi-node architecture applies. Key considerations include disk sizing (dependent on alert volume and retention), JVM heap allocation (half of available RAM, up to 32 GB), and enabling memory locking to prevent swapping.

On-Premises



6. Integration with the Broader Platform

The Indexer integrates with the rest of the Wazuh ecosystem in three key ways. First, the **Wazuh Server** forwards analyzed events via Filebeat; additionally, the Content Manager plugin synchronizes detection content (rules, decoders, IoCs) from the Wazuh CTI API back to the Server. Second, the **Wazuh Dashboard** — an OpenSearch Dashboards fork

— queries the Indexer’s REST API to visualize and search indexed data. Third, data stored in the Indexer can be pulled by **Logstash** and forwarded to **third-party platforms** such as OpenSearch, Elasticsearch, Splunk, or Amazon Security Lake. In this flow, the Indexer itself does not push data — Logstash queries the Indexer’s indices and ships the results onward. There is also an alternative Server-side integration where Logstash reads the `alerts.json` file directly from the Wazuh Server, bypassing the Indexer entirely.

7. Conclusion

The Wazuh Indexer is not the most visible component in the stack — it does not analyze traffic, fire alerts, or present dashboards. But it is arguably the most critical. If the Indexer is down, the Dashboard shows nothing and historical investigation becomes impossible. What makes it effective is the combination of near real-time search, automatic shard distribution, and the purpose-built plugins that tailor it specifically for security monitoring. When properly sized and backed by ISM lifecycle policies, it provides a reliable foundation for the entire Wazuh ecosystem.